

■ Design Top-K / Trending Topics System

System Design Interview | Counting & Ranking

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional

Track frequency of events (search queries, hashtags, videos); Return top-K most frequent items; Real-time updates; Time-window based (last 1h/24h)

Non-Functional

Handle millions of events/sec; Low latency query (<100ms); Approximate results acceptable; Horizontally scalable

Scale

Twitter: 500M tweets/day; YouTube: billions of views; Google: 8.5B searches/day; exact counting infeasible at this scale

■ Architecture Overview

- Event producers → Kafka topics → Stream Processors (Flink/Spark Streaming) → Aggregated counts → Redis/DB
- Two-layer aggregation: local aggregation per partition (reduce data) → global merge (final top-K)
- Time windowing: sliding windows (last N minutes) or tumbling windows (fixed time buckets)
- Result storage: Redis sorted set (ZRANGEBYSCORE) for real-time top-K queries; $O(\log N)$ per update

■ Approximate Counting Algorithms

- **Count-Min Sketch:** 2D array of counters with multiple hash functions; update: increment d hash positions; query: take minimum; $O(1)$ update/query; ~95% accurate with 1-5% overcount
- **HyperLogLog:** Estimate unique item count (cardinality) from hashed values; 1.5% error; uses only 12KB for billions of elements; Redis HLL built-in
- **Lossy Counting:** Approximate frequency of items in streaming data; guarantee: items with $\text{freq} > N \times \epsilon$ counted; sublinear memory
- **Heavy Hitters (Space Saving):** Exactly tracks top-K candidates; when buffer full: evict min-count item; add new item with that count+1; widely used in practice

■ Redis Sorted Sets for Top-K

- ZADD trending hashtag 1 INCREMENT (atomic increment); ZREVRANGE trending 0 9 WITHSCORES (top 10)
- Time decay: multiply all scores by decay factor periodically, or use separate keys per time bucket
- Sliding window: maintain separate sorted sets per minute; merge last 60 for 1-hour window; expire old sets with TTL
- Sharding: partition by first char or hash to distribute load; merge top-K from each shard for final answer

■ Stream Processing with Flink

- Event time processing: use event timestamp not processing time; handle late arrivals with watermarks

- Tumbling window: fixed non-overlapping 1-min buckets; end-of-window triggers top-K computation
- Sliding window: overlapping windows (every 10s, cover last 5min); more granular but higher compute
- Operator: countByKey → reduce → topK using min-heap; emit top-K per window to output topic

■ Exact vs Approximate

Exact Count (HashMap)	Approximate (Count-Min Sketch)
✓ 100% accurate	✓ ~95-99% accurate
✓ Memory: O(unique items)	✓ Fixed memory (KB-MB)
✓ Infeasible at billions scale	✓ Any scale
✓ OK for small cardinality	✓ Overcount only, never undercount

■ Real-World Implementations

- Twitter trending: sliding window + Heavy Hitters algorithm + geographic/language filtering; updated every 30s
- YouTube trending: not pure view count — weights recency, growth rate, engagement (likes/comments) + geo-boost
- Google search trending: uses internal distributed counting + ML to detect emerging trends vs existing popular terms
- Redis built-in: ZRANGEBYSCORE for time-based top-K; sorted sets handle 1M+ keys efficiently